

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	29 September 2026
Team ID	Team No: 05
Project Name	AI WeatherWise
Maximum Marks	4 Marks

Technical Architecture:

The deliverable shall include the architectural information as per Table-1 and Table-2. AI WeatherWise is a headless RESTful backend developed using Node.js and Express.js. It uses MongoDB with Mongoose for data persistence, OpenWeatherMap for weather data, and Google Gemini AI for weather insights and recommendations when the external AI service is available.

Architecture Flow
API Client → Node.js + Express.js REST API → MongoDB Atlas / OpenWeatherMap API / Google Gemini AI
Authentication & Security: JWT, bcryptjs, CORS and dotenv
AI Recommendation: Gemini AI when available; fallback response when the external AI service is unavailable

The technology stack identifies the application logic, database, external interfaces, AI service, security mechanisms, and infrastructure used by the project.

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	The project is a headless REST API. API requests are sent through an API client for development and testing.	REST API / Thunder Client
2.	Application Logic-1	Core server and REST API request/response handling.	Node.js, Express.js
3.	Application Logic-2	User registration/login, password hashing and token-based authentication.	JWT, bcryptjs
4.	Application Logic-3	Weather retrieval and AI recommendation/insight processing with fallback handling.	OpenWeatherMap API, Google Gemini AI
5.	Database	Stores user and favorite-location data using Mongoose schemas.	MongoDB, Mongoose
6.	Cloud Database	Cloud-hosted MongoDB database used by the backend.	MongoDB Atlas
7.	File Storage	No separate file-storage service is specified for the backend.	Not used / Local project files
8.	External API-1	Provides real-time weather data used by the weather endpoint.	OpenWeatherMap API
9.	External API-2	Provides AI-based weather insights/recommendations when available.	Google Gemini API
10.	Machine Learning Model	AI model interface for natural-language weather insights and recommendations.	Google Gemini AI
11.	Infrastructure (Server / Cloud)	Backend runs as a Node.js server during development; database and external services are cloud-based.	Local Windows server, Node.js, MongoDB Atlas

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Open-source technologies and libraries used to implement the backend.	Node.js, Express.js, Mongoose, bcryptjs, jsonwebtoken
2.	Security Implementations	Authentication, password protection, cross-origin handling and environment-based secret configuration.	JWT, bcryptjs, CORS, dotenv

S.No	Characteristics	Description	Technology
3.	Scalable Architecture	The backend is modular, separating routes, controllers, middleware, models and services. This supports adding endpoints or scaling backend instances without redesigning the whole application.	Node.js, Express.js, modular REST architecture
4.	Availability	The recommendation endpoint includes fallback handling when the external AI service is unavailable. Weather retrieval remains dependent on the external weather API.	Fallback logic, Express.js, OpenWeatherMap API
5.	Performance	The application uses REST endpoints and a modular service layer. No caching or CDN mechanism is specified in the project implementation.	Express.js, Mongoose

References:

<https://c4model.com/>

<https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>